

Code Explanation

Field Calculator Module Explanation

The provided Python module, `field\_calculator.py`, is designed to standardize and calculate fields for a specific data processing task, likely related to the 10K file. It utilizes Apache Spark for data processing and provides various functions to standardize string and date fields, translate codes, and apply field-specific calculations.

Overview of the Code

This module includes several key functions: `standardize\_string\_field`, `standardize\_date\_field`, `translate\_code`, `apply\_field\_calculations`, and `generate\_file\_name`. These functions work together to clean, standardize, and transform data according to predefined rules and configurations.

Step 1: Importing Necessary Modules and Initializing Logger

The code begins by importing necessary modules from PySpark and Python's standard library. It also initializes a logger for logging purposes.

```
import logging
from typing import Dict, Any, List
from pyspark.sql import DataFrame
from pyspark.sql.functions import (
    col, lit, when, concat, lpad, rpad, trim,
    substring, regexp_replace, date_format, coalesce,
    upper, lower, expr
)
logger = logging.getLogger( name )
```

Step 2: Defining the `standardize\_string\_field` Function

This function takes a DataFrame, a field name, a maximum length, a padding character, and an alignment parameter. It trims the field, removes special characters, truncates or pads it to the specified length, and aligns it accordingly.

```
def standardize_string_field(df: DataFrame, field_name: str, max_length: int,
                             pad_char: str = ' ', align: str = 'left')
    -> DataFrame:
    cleaned_df = df.withColumn(
        field_name,
        regexp_replace(trim(col(field_name)), r'^x20-x7E', '')
    )
    if align.lower() == 'left':
        return cleaned_df.withColumn(
            field_name,
            rpad(substring(col(field_name), 1, max_length), max_length, pad_char)
        )
    else:
        return cleaned_df.withColumn(
            field_name,
            lpad(substring(col(field_name), 1, max_length), max_length, pad_char)
```

```
)
```

Step 3: Defining the `standardize\_date\_field` Function

This function standardizes a date field to a specified output format. It optionally accepts an input format for the date conversion.

```
def standardize_date_field(df: DataFrame, field_name: str,
                           input_format: str = None,
                           output_format: str = 'yyyyMMdd') -> DataFrame:
    if input_format:
        return df.withColumn(
            field_name,
            date_format(col(field_name).cast("timestamp"), output_format)
        )
    else:
        return df.withColumn(
            field_name,
            date_format(col(field_name), output_format)
        )
```

Step 4: Defining the `translate\_code` Function

This function translates codes in a specified field based on a provided mapping dictionary.

```
def translate_code(df: DataFrame, field_name: str, code_map: Dict[str, str],
                  default_value: str = '') -> DataFrame:
    case_expr = when(col(field_name).isNull(), lit(default_value))
    for input_code, output_code in code_map.items():
        case_expr = case_expr.when(col(field_name) == input_code, lit(output_code))
    case_expr = case_expr.otherwise(lit(default_value))
    return df.withColumn(field_name, case_expr)
```

Step 5: Applying Field Calculations with `apply\_field\_calculations`

This function applies various field calculations and standardizations to the input DataFrame. It includes translating gender codes, standardizing string and date fields according to predefined configurations.

```
def apply_field_calculations(df: DataFrame) -> DataFrame:
    logger.info("Starting field calculations")
    result_df = df
    gender_map = {'M': '1', 'F': '2', 'U': '3'}
    result_df = translate_code(result_df, "BENEFICIARY_GENDER", gender_map, '3')
    # Example configurations for string and date fields
    string_fields = [{"name": "Field_1", "length": 50, "align": "left"}]
    date_fields = [{"name": "Field_10", "input_format": None, "output_format": "yyyyMMdd"}]
```

```

    for field_config in string_fields:
        result_df = standardize_string_field(result_df, field_config["name"], field_config["length"], ' ', field_config["align"])
    for field_config in date_fields:
        result_df = standardize_date_field(result_df, field_config["name"], field_config["input_format"], field_config["output_format"])
    logger.info("Field calculations completed")
    return result_df

```

Step 6: Generating File Name with `generate\_file\_name`

This function generates a file name based on the output directory, file prefix, batch ID, sequence number, and file extension, and adds it as a new column to the DataFrame.

```

def generate_file_name(df: DataFrame, output_dir: str, file_prefix: str,
                      batch_id: str, file_ext: str, sequence_num: int) -> DataFrame:
    file_name = f"{output_dir}/{file_prefix}_{batch_id}_{sequence_num:03d}.{file_ext}"
    return df.withColumn("FileName", lit(file_name))

```